uses the term *skills*. Therefore, throughout this chapter, we'll use *skills* and *plugins* to mean the same thing.

SK is a useful tool for managing multiple plugins (actions for agents) and, as we'll see later, can also assist with memory and planning tools. For this chapter, we'll focus on the actions/plugins. In the next section, we look at how to get started using SK.

5.3.1 Getting started with SK semantic functions

SK is easy to install and works within Python, Java, and C#. This is excellent news as it also allows plugins developed in one language to be consumed in a different language. However, you can't yet develop a native function in one language and use it in another.

We'll continue from where we left off for the Python environment using the `chapter_4` workspace in VS Code. Be sure you have a workspace configured if you want to explore and run any examples.

Listing 5.8 shows how to install SK from a terminal within VS Code. You can also install the SK extension for VS Code. The extension can be a helpful tool to create plugins/skills, but it isn't required.

Listing 5.8 Installing Semantic Kernel

```
pip uninstall semantic-kernel     #1

git clone https://github.com/microsoft/semantic-kernel.git     #2

cd semantic-kernel/python     #3

pip install -e .     #4
```

#1 Uninstalls any previous installations of SK
#2 Clones the repository to a local folder
#3 Changes to the source folder
#4 Installs the editable package from the source folder

Once you finish the installation, open `SK_connecting.py` in VS Code. Listing 5.9 shows a demo of running an example quickly through SK. The example creates a chat completion service using either OpenAI or Azure OpenAI.